

DEPTH THROUGH RECURRENCE: LOOPED TRANSFORMERS FOR FLOW-MATCHING TTS

Jiabao Ai^{1,2} Peng Han^{1,2} Yuchen Song^{1,2} Zhengjun Yue^{1,2}

¹Shenzhen Loop Area Institute, Shenzhen, China

²The Chinese University of Hong Kong, Shenzhen

ABSTRACT

We study how to organize Transformer depth through recurrence in flow-matching text-to-speech, varying the amount, order, and placement of weight reuse. Seven layouts perform 18 block calls per network evaluation under a common training objective and sampler. On Seed-TTS and LibriSpeech-PC, SEQUENCE applies each of nine blocks twice consecutively, retaining competitive intelligibility, speaker similarity, and predicted speech quality at 32 sampling steps with 47.1% fewer parameters than the unshared baseline. Cycling six blocks three times further reduces model size but raises 32-step word error rates relative to cycling nine blocks twice. At matched parameter counts and executed depth, reuse order and sharing position produce different quality trade-offs. These comparisons depend on sampling budget: Prefix and Suffix have similar 32-step word error rates, but Suffix is worse by 3.44 and 5.97 percentage points at four steps on the two datasets, respectively. Only Middle ranks first or second in mean word error rate at 32 and four steps on both datasets. These results show that the organization of recurrent computation affects synthesis quality, and that reuse layouts should be selected for both the sampling budget and the quality dimensions of interest.

Index Terms— text-to-speech, flow matching, weight sharing, looped transformers

1. INTRODUCTION

Flow-matching TTS systems such as Voicebox [1], E2 TTS [2] and F5-TTS [3] repeatedly evaluate a velocity network during numerical integration [4]. Computation therefore has two dimensions: network depth and sampling steps. Reusing Transformer blocks over evolving hidden states preserves executed depth with fewer unique parameters. How should this recurrence be organized when the velocity network is itself repeatedly evaluated during sampling?

Cross-layer sharing is established in Universal Transformers [5] and ALBERT [6]; Subformer keeps the first and last layers unshared [7]. Reuse order matters in translation [8] and language-model pre-training [9], while recent looped language models study recurrent computation for efficiency and latent reasoning [10, 11, 12, 13]. In visual generation, ELT adds intra-loop self-distillation [14], and Fixed Point Diffusion Models combine implicit recurrence with unshared boundary layers [15]. We study how these reuse principles interact with sampling budget and speech-quality criteria in flow-matching TTS.

We compare seven layouts with 18 block calls per network evaluation under a common objective, sampler, width and training procedure (Fig. 1). Baseline, CYCLE 9×2 and CYCLE 6×3 vary sharing amount; CYCLE 9×2 and SEQUENCE 9×2 compare

whole-stack and adjacent-block reuse at matched size; Prefix, Middle and Suffix vary the placement of a repeated six-block segment. Audio samples are available online.¹

On Seed-TTS and LibriSpeech-PC, SEQUENCE retains competitive 32-step quality with 47.1% fewer parameters than Baseline. Cycling six blocks three times instead of nine twice worsens 32-step intelligibility. Reuse order changes the trade-offs among intelligibility, speaker similarity and predicted quality. Placement interacts with sampling budget: Prefix and Suffix have close 32-step word error rates (WER) but diverge at four steps. Only Middle ranks first or second in mean WER across both budgets and datasets. Its unshared early calls are more removal-sensitive than its tested shared interior. These findings guide layout selection by sampling budget and quality priorities.

2. RECURRENT DEPTH FOR FLOW-MATCHING TTS

The velocity network $v_\theta(x_t, t, c)$ conditions on noisy speech x_t , flow time t , and text and reference-speech context c . Within one network evaluation, a shared m -block segment F_θ updates its hidden state at visit k as

$$h^{(k+1)} = F_\theta(h^{(k)}; t, c). \quad (1)$$

Weights, flow time t , and conditioning remain fixed across visits; the hidden state is not reset. No visit-index embedding or auxiliary supervision is used.

Baseline. The unshared model has 18 unique blocks and 158.0M parameters.

Full recurrence. CYCLE 9×2 executes $1, \dots, 9, 1, \dots, 9$, revisiting the whole stack; SEQUENCE 9×2 executes $1, 1, 2, 2, \dots, 9, 9$, immediately reapplying each block. Both have nine unique blocks and 83.6M parameters. CYCLE 6×3 cycles six blocks three times and has 58.8M parameters.

Partial recurrence. A layout has p unshared blocks, an m -block segment cycled K times, then q unshared blocks, giving $D = p + mK + q$ executed calls. Prefix, Middle, and Suffix respectively use $(p, m, K, q) = (0, 6, 2, 6)$, $(3, 6, 2, 3)$, and $(6, 6, 2, 0)$. Each has 12 unique blocks, 108.4M parameters, and $D = 18$; only the shared segment’s position changes.

Sampling budget. Recurrence operates within each network evaluation; sampling repeatedly evaluates the network at successive flow times. With executed depth D , N sampling steps, and two classifier-free-guidance (CFG) branches [16], the block-call budget is

$$B = 2ND. \quad (2)$$

¹<https://jiabaoai67.github.io/loop-f5/>

Layout	18 block calls →	Unique	Params
Baseline	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18	18	158.0M
CYCLE 9×2	1 2 3 4 5 6 7 8 9 1 2 3 4 5 6 7 8 9	9	83.6M
SEQUENCE 9×2	1 1 2 2 3 3 4 4 5 5 6 6 7 7 8 8 9 9	9	83.6M
CYCLE 6×3	1 2 3 4 5 6 1 2 3 4 5 6 1 2 3 4 5 6	6	58.8M
Prefix	1 2 3 4 5 6 1 2 3 4 5 6 7 8 9 10 11 12	12	108.4M
Middle	1 2 3 4 5 6 7 8 9 4 5 6 7 8 9 10 11 12	12	108.4M
Suffix	1 2 3 4 5 6 7 8 9 10 11 12 7 8 9 10 11 12	12	108.4M

 Not shared
 1st use
 2nd use
 3rd use
Same ID within a row = same weights

Fig. 1. Weight-sharing layouts with 18 block calls per network evaluation. Repeated IDs within a row share weights; darker shades indicate later uses. Gray blocks are unshared.

Table 1. Quality and inference memory at 500k updates. Quality means over four/three inference seeds on Seed-TTS/LibriSpeech-PC; bold marks the best displayed means at each sampling budget. WER: percent. Infer. mem.: peak PyTorch-allocated memory relative to Baseline (H100, fp32, batch 1, 32 steps; vocoder included).

(a) 32 sampling steps		Resources		Seed-TTS test-en			LibriSpeech-PC		
Sharing	Layout	Params (M)	Infer. mem. (%)	WER↓	SIM-o↑	UTMOS↑	WER↓	SIM-o↑	UTMOS↑
None	Baseline	158.0	100.0	2.23	0.567	3.68	2.29	0.592	3.89
	CYCLE 9×2	83.6	65.5	2.60	0.562	3.90	2.14	0.600	4.06
Full	SEQUENCE 9×2	83.6	65.5	2.03	0.573	3.75	2.26	0.603	3.93
	CYCLE 6×3	58.8	54.0	2.96	0.557	3.86	2.65	0.593	4.06
Partial	Prefix	108.4	77.0	2.36	0.563	3.88	2.24	0.569	4.09
	Middle	108.4	77.0	2.04	0.566	3.84	2.07	0.583	4.04
	Suffix	108.4	77.0	2.19	0.570	3.91	2.10	0.593	4.02

(b) 4 sampling steps		Seed-TTS test-en			LibriSpeech-PC		
Sharing	Layout	WER↓	SIM-o↑	UTMOS↑	WER↓	SIM-o↑	UTMOS↑
None	Baseline	10.59	0.498	2.85	17.81	0.433	2.23
	CYCLE 9×2	9.37	0.496	3.08	11.36	0.444	2.59
Full	SEQUENCE 9×2	9.32	0.498	3.07	11.46	0.428	2.56
	CYCLE 6×3	11.56	0.495	3.03	16.68	0.456	2.80
Partial	Prefix	6.87	0.495	3.24	11.20	0.415	2.81
	Middle	8.23	0.496	3.10	10.83	0.445	2.70
	Suffix	10.31	0.499	3.07	17.17	0.423	2.41

Conditional and unconditional branches count separately, even when batched. All seven layouts retain $D = 18$, giving $B = 36N$. Changing N varies numerical integration of a fixed velocity network. Block calls quantify computation; latency is measured separately.

3. EXPERIMENTAL SETUP

3.1. Training and controls

All seven layouts use a modified F5-TTS Small implementation [3] (width 768) and are trained from scratch on LibriTTS train-clean-100/360 and train-other-500 [17]. The common configuration uses AdamW with peak learning rate 7.5×10^{-5} , 20k warm-up updates

and linear decay to zero at update 588,931, gradient clipping at 1.0, bf16 mixed precision, up to 307,200 mel frames per update (eight GPUs, accumulation of two), per-rank CFG dropout, and nominal seed 666. Each layout has one training run; comparisons use its 500k-update exponential moving average (EMA) weights.

3.2. Evaluation

Seed-TTS test-en [18, 19] has 1,088 utterances. For LibriSpeech-PC, we use the F5-TTS subset [3, 20]: 1,127 utterances from 39 speakers. We use Euler sampling, CFG 2.0, sway -1 , batch size one and fp32; target durations follow the F5-TTS text-length rule, and Vocos [21] produces waveforms. We compare 32 and four steps, with intermediate budgets tracing changes in layout performance.

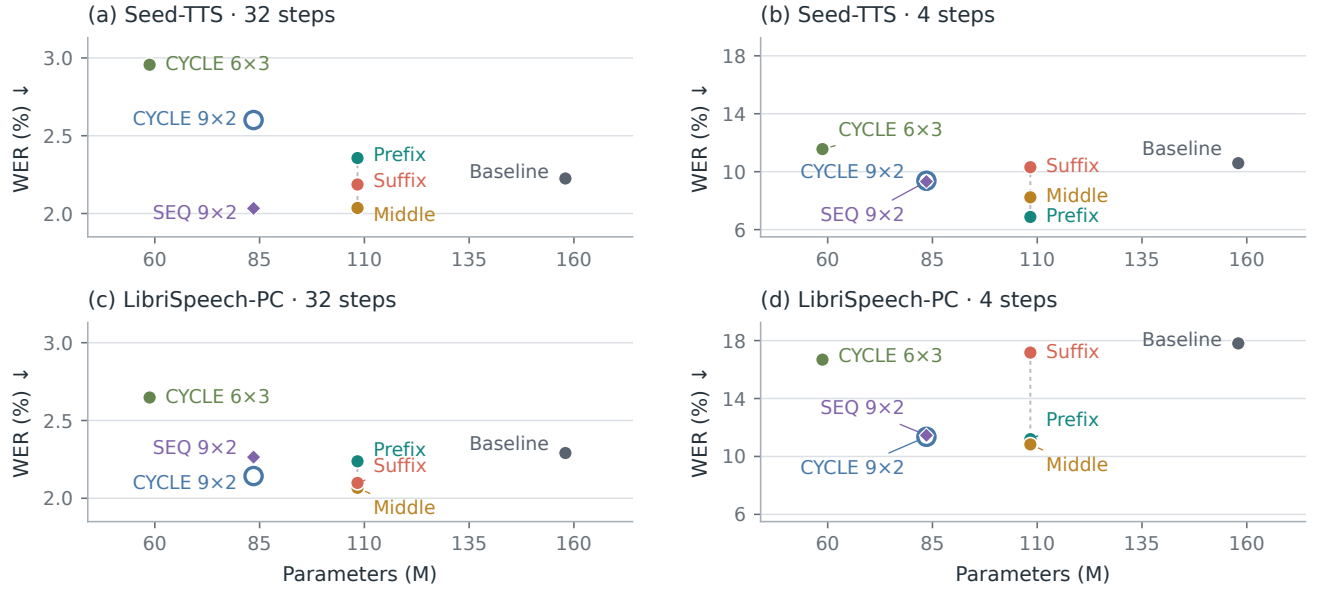


Fig. 2. WER versus parameter count at 32 and four steps, using the multi-seed means from Table 1. Vertical guides connect partial-loop layouts. SEQ: SEQUENCE.

WER measures intelligibility using mean per-utterance Whisper large-v3 error rate [22], as in the F5-TTS evaluation code. **SIM-o** is cosine similarity between prompt and output embeddings from a WavLM-Large/ECAPA-TDNN speaker verifier [23], using 16-kHz audio and per-waveform normalization. **UTMOS22 strong** [24] predicts speech quality; it is not a listening-test score.

3.3. Uncertainty and resource measurement

Main-table quality scores average four/three inference seeds on Seed-TTS/LibriSpeech-PC. We first average each utterance over common seeds, then bootstrap paired differences over 666 reference-prompt clusters or 39 speaker clusters, respectively, using 20,000 resamples. The unadjusted 95% intervals describe evaluation uncertainty conditional on trained checkpoints, not training-seed variance.

Resources are measured on H100 with fp32, batch one and 32-step Seed-TTS synthesis. Peak memory is PyTorch-allocated and includes the vocoder. Sampling real-time factor (RTF) is total sampling time divided by total generated audio duration, excluding vocoding.

3.4. Diagnostic protocol

For each call of the 500k EMA models, we measure the relative update $\|\Delta h\|/\|h\|$ on target frames [25], and the cosine between two visits’ updates to a shared block. We use 128 LibriSpeech-PC utterances and ground-truth/noise interpolants at seven flow times from 0 to 0.9 in both CFG branches (14 settings). For each block-setting pair, statistics are computed per utterance and then averaged. Call removal sets one call’s output to its input at every Euler step in both branches [26], on Seed-TTS with inference seed 666 and 16 steps, without retraining.

4. RESULTS AND DISCUSSION

4.1. Parameter efficiency at matched executed depth

SEQUENCE uses 83.6M parameters versus Baseline’s 158.0M, a 47.1% reduction, with the same 18 block calls. At 32 steps, its WER is 2.03/2.26% on Seed-TTS/LibriSpeech-PC versus Baseline’s 2.23/2.29% (Table 1). The paired differences are -0.19 percentage points (pp), 95% CI $[-0.48, +0.06]$, and -0.03 pp $[-0.24, +0.16]$. SIM-o and UTMOS means are higher on both datasets, with paired intervals above zero. These comparisons support competitive objective quality at a smaller parameter budget; they do not establish equivalence.

The benefit has limits as sharing increases. CYCLE 9×2 has 32-step WER of 2.60/2.14%; CYCLE 6×3 saves another 24.8M parameters but raises WER to 2.96/2.65% and lowers SIM-o on both datasets. Predicted quality does not decline uniformly: LibriSpeech-PC UTMOS is 4.06 for both. WER is not monotonic in parameter count (Fig. 2).

CYCLE 9×2 and SEQUENCE 9×2 reduce peak allocated memory from 877 to 574 MB (34.5%); CYCLE 6×3 and the partial loops use 474 and 675 MB. Sampling RTF spans 0.1426–0.1440 across all seven layouts. Parameter storage and peak allocated memory fall, with comparable observed sampling time.

4.2. Reuse order changes the quality trade-off

At matched size and depth, SEQUENCE lowers 32-step Seed-TTS WER from CYCLE’s 2.60 to 2.03, a paired change of -0.57 pp $[-0.80, -0.35]$, but lowers UTMOS from 3.90 to 3.75. WER rises from 2.14 to 2.26 on LibriSpeech-PC; its paired interval includes zero. SEQUENCE has the highest 32-step SIM-o means on both datasets, but lower UTMOS than CYCLE, with paired intervals below zero. Its Seed-TTS WER is nearly tied with Middle (2.033 versus 2.036). At four steps, CYCLE and SEQUENCE have close

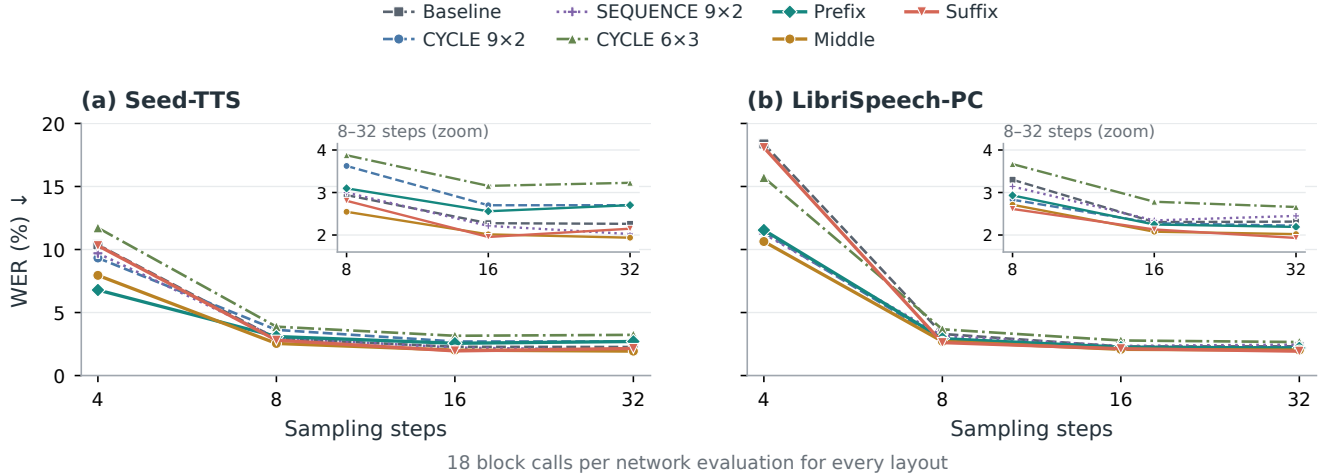


Fig. 3. WER versus sampling steps for all seven layouts. Curves use one fixed inference seed per dataset (Seed-TTS: 666; LibriSpeech-PC: 0). Insets show 8–32 steps.

WER: 9.37 versus 9.32 on Seed-TTS, and 11.36 versus 11.46 on LibriSpeech-PC. Reuse order changes the quality trade-off without producing a universal winner.

4.3. Placement interacts with the sampling budget

At 32 steps, Prefix and Suffix differ by only 0.17/0.14 pp in WER, with Suffix slightly lower. At four steps, Suffix instead exceeds Prefix by 3.44 pp, 95% CI [2.77, 4.14], and 5.97 pp [3.69, 8.70]. From 32 to four steps, the Suffix-minus-Prefix gap therefore grows by 3.61/6.11 pp, with both paired intervals above zero. Thus identical model size and executed depth conceal a substantial difference at the most aggressive sampling budget tested.

The intermediate budgets localize this difference (Fig. 3). In the fixed-seed curves, Suffix-minus-Prefix WER changes from +3.52/+6.55 pp at four steps to $-0.29/-0.32$ pp at eight. The mean ordering reverses, although the Seed-TTS eight-step interval includes zero. Prefix therefore does not lead throughout the low-step range. The max-min WER spread across the seven layouts shrinks from 4.93/7.74 pp at four steps to 1.20/0.70 pp at 16 steps and 1.29/0.73 pp at 32 steps.

The four-step gap includes more high-error outputs. Across inference seeds, moving from Prefix to Suffix raises the fraction of individual generations with WER above 25% from 7.54% to 12.89% on Seed-TTS and from 12.51% to 24.82% on LibriSpeech-PC; 36 of 39 LibriSpeech-PC speakers have higher mean four-step WER. At 32 steps, the corresponding Prefix/Suffix rates are 1.40%/1.42% on Seed-TTS and 0.59%/0.53% on LibriSpeech-PC.

Quality criteria also change the comparison. At four steps, Prefix exceeds Suffix in UTMOS by 0.164/0.396, with both paired intervals above zero, but has the lowest LibriSpeech-PC SIM-o among all layouts. At 32 steps, Suffix exceeds Prefix in SIM-o by 0.0064/0.0238, with both intervals above zero.

Among partial loops, Middle has the lowest 32-step WER on both datasets and the lowest four-step WER on LibriSpeech-PC. Across all seven layouts, only Middle ranks first or second in mean WER at both budgets on both datasets. Its WER is below Baseline’s in all four conditions, with paired intervals below zero except on Seed-TTS at 32 steps. Middle does not lead in SIM-o or UTMOS.

4.4. Update structure and call sensitivity

Baseline’s first-call mean relative update is 1.6–2.1 across the 14 settings. Removing call 1, 3, 4 or 5 raises WER from 2.28% to 11.12–102.24%; removing any call after the fifth changes it by at most 4.64 pp. Middle’s first two calls are unshared and have the largest mean relative updates in every setting; removing either gives 28.84% or 99.97% WER, versus 2.02% unmodified. Removing shared calls 4–14 individually (call 15 untested) changes WER by at most 1.09 pp. Prefix’s first-call mean relative update is at most 0.47, yet removing the second visit of block 4 gives 61.27% WER. The mean of the last three relative updates is 1.60–2.85 times the median over calls 6–13 for Baseline and Middle, but 0.84–1.35 times for Suffix, which ends on second visits. Among partial loops, only Middle combines large early relative updates with an increase near the output; its boundary blocks are unshared.

Reuse order changes how two visits of a block relate. In SEQUENCE, mean cosines between each block’s two visit updates are positive across all 126 block-setting pairs (+0.42 to +0.94). In CYCLE 9×2 , block 1 is reapplied to the whole-stack output; its visit updates have negative mean cosine across all 14 settings (-0.61 to -0.33). Averaged over blocks and settings, the visit cosine is 0.71 for SEQUENCE, 0.35 for CYCLE, and highest among partial loops for Middle (0.62; Prefix 0.47, Suffix 0.52).

5. CONCLUSION

At matched executed depth, our looped flow-matching TTS models retain competitive quality with fewer parameters. Cycling six blocks three times instead of nine twice worsens 32-step intelligibility at 500k updates. Reuse order changes 32-step quality trade-offs but barely affects four-step WER; placement changes four-step WER much more than 32-step WER. Middle ranks first or second in mean WER across all four dataset-budget conditions. Layout choice depends on model size, sampling budget and quality priorities.

The evidence covers one model scale and English objective evaluation. Future work will add listening tests.

6. REFERENCES

- [1] Matthew Le, Apoorv Vyas, Bowen Shi, Brian Karrer, Leda Sari, Rashel Moritz, et al., “Voicebox: Text-Guided Multilingual Universal Speech Generation at Scale,” in *Proc. NeurIPS*, 2023.
- [2] Sefik Emre Eskimez, Xiaofei Wang, Manthan Thakker, Canrun Li, Chung-Hsien Tsai, Zhen Xiao, et al., “E2 TTS: Embarrassingly Easy Fully Non-Autoregressive Zero-Shot TTS,” in *Proc. IEEE SLT*, 2024.
- [3] Yushen Chen, Zhikang Niu, Ziyang Ma, Keqi Deng, Chunhui Wang, Jian Zhao, Kai Yu, and Xie Chen, “F5-TTS: A Fairytaler that Fakes Fluent and Faithful Speech with Flow Matching,” in *Proc. ACL*, 2025.
- [4] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le, “Flow Matching for Generative Modeling,” in *Proc. ICLR*, 2023.
- [5] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser, “Universal Transformers,” in *Proc. ICLR*, 2019.
- [6] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut, “ALBERT: A Lite BERT for Self-supervised Learning of Language Representations,” in *Proc. ICLR*, 2020.
- [7] Machel Reid, Edison Marrese-Taylor, and Yutaka Matsuo, “Subformer: Exploring Weight Sharing for Parameter Efficiency in Generative Transformers,” in *Findings of EMNLP*, 2021.
- [8] Sho Takase and Shun Kiyono, “Lessons on Parameter Sharing across Layers in Transformers,” in *Proc. SustaiNLP Workshop (ACL)*, 2023.
- [9] Zitian Gao, Yilong Chen, Yihao Xiao, Xinyu Yang, Ran Tao, Joey Zhou, and Bryan Dai, “Loop the Loopies!,” *arXiv preprint arXiv:2607.16051*, 2026.
- [10] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, et al., “Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach,” in *Proc. NeurIPS*, 2025.
- [11] Yeskendir Koishekenov, Aldo Lipani, and Nicola Cancedda, “Encode, Think, Decode: Scaling Test-Time Reasoning with Recursive Latent Thoughts,” *arXiv preprint arXiv:2510.07358*, 2025.
- [12] Amr Hegazy, Amr Alanwar, and Mostafa Elhoushi, “Gated Recurrent Transformers: Expressive Depth through Recurrent Modulation,” *arXiv preprint arXiv:2608.15062*, 2026.
- [13] Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li, “SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers,” *arXiv preprint arXiv:2609.01343*, 2026.
- [14] Sahil Goyal, Swayam Agrawal, Gautham Govind Anil, Prateek Jain, Sujoy Paul, and Aditya Kusupati, “ELT: Elastic Looped Transformers for Visual Generation,” in *Proc. ECCV*, 2026.
- [15] Xingjian Bai and Luke Melas-Kyriazi, “Fixed Point Diffusion Models,” in *Proc. CVPR*, 2024.
- [16] Jonathan Ho and Tim Salimans, “Classifier-Free Diffusion Guidance,” *arXiv preprint arXiv:2207.12598*, 2022.
- [17] Heiga Zen, Viet Dang, Rob Clark, Yu Zhang, Ron J. Weiss, Ye Jia, Zhifeng Chen, and Yonghui Wu, “LibriTTS: A Corpus Derived from LibriSpeech for Text-to-Speech,” in *Proc. Interspeech*, 2019.
- [18] Philip Anastassiou, Jiawei Chen, Jitong Chen, Yuanzhe Chen, Zhuo Chen, Ziyi Chen, et al., “Seed-TTS: A Family of High-Quality Versatile Speech Generation Models,” *arXiv preprint arXiv:2406.02430*, 2024.
- [19] ByteDance Seed, “seed-tts-eval: objective evaluation set for Seed-TTS,” <https://github.com/BytedanceSpeech/seed-tts-eval>, 2024.
- [20] Aleksandr Meister, Matvei Novikov, Nikolay Karpov, Evelina Bakhturina, Vitaly Lavrukhin, and Boris Ginsburg, “LibriSpeech-PC: Benchmark for Evaluation of Punctuation and Capitalization Capabilities of end-to-end ASR Models,” in *Proc. IEEE ASRU*, 2023.
- [21] Hubert Siuzdak, “Vocos: Closing the gap between time-domain and Fourier-based neural vocoders for high-quality audio synthesis,” in *Proc. ICLR*, 2024.
- [22] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proc. ICML*, 2023.
- [23] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, et al., “WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [24] Takaaki Saeki, Detai Xin, Wataru Nakata, Tomoki Koriyama, Shinnosuke Takamichi, and Hiroshi Saruwatari, “UTMOS: UTokyo-SaruLab System for VoiceMOS Challenge 2022,” in *Proc. Interspeech*, 2022.
- [25] Stanisław Jastrzębski, Devansh Arpit, Nicolas Ballas, Vikas Verma, Tong Che, and Yoshua Bengio, “Residual Connections Encourage Iterative Inference,” in *Proc. ICLR*, 2018.
- [26] Andreas Veit, Michael J. Wilber, and Serge Belongie, “Residual Networks Behave Like Ensembles of Relatively Shallow Networks,” in *Proc. NeurIPS*, 2016.